

TCMS in trigger-service (rebuild from scratch)

Problem Statement

The QA team's test cases live in spreadsheets and in people's heads. Automated tests carry an id in code (`test_case_id`), but nothing records what each test checks, which business rule it covers, which Jira ticket it belongs to, or lets anyone run one case by id. AI agents cannot read business rules because they are not written down anywhere. Compliance (SOC 2) has no per-requirement evidence of testing. Earlier TCMS attempts never reached the team.

Evidence

- Test cases are in spreadsheets "and sometimes in someone's head" (owner statement, 2026-10-05).
- Business rules "don't live anywhere for now" (owner statement).
- Three earlier TCMS builds in this repo were never merged to `master: chore/tcms-alignment` (local only, 16 commits), `feat/tcms-m0-m1-audit-pack`, `fix/tcms-forms-and-ci`. The owner says they "did not reach the light" for lack of a dedicated builder.
- The old build seeded about 3,565 case rows on 2026-09-09; only 46 had steps (`_tcms_bridge` response files, per exploration).
- Test ids are not unique today. Rewards suite: ~1,493 distinct ids on 1,661 collected tests; 80 ids appear on more than one test; 37 tests have no id; 61 id literals are prefixes of other ids; bare `TC001/TC002` are reused across files (stored collect-only of 2026-09-23, `platform_svc_test-suite/.verify-`

```
artifacts/test-pr/pr-49/20260923-214338/collect-head.json).
```

- There is no run-by-id feature. Selection is a substring `-k` filter (`platform_svc_test-suite/main.py:153`).

Proposed Solution

Build a Test Case Management System inside trigger-service: a central, versioned repository of test cases keyed by the existing unique test case id. Cases are imported from the rewards test source, enriched by QA with steps and the business rule, linked to Jira tickets, run in isolation by id through trigger-service, and shown with their execution history. Compliance gets a read-only view and CSV export per Jira ticket. AI agents get a read-only JSON export. We build in trigger-service (owner's decision) because it already launches runs and stores results, so run-by-id and execution evidence come without a second system.

Key Hypothesis

We believe a central TCMS in trigger-service, keyed by unique test case id, will let QA find, review, run and evidence every automated rewards test. We'll know we're right when, by day 20, 100% of the ids found by the source scan are in the TCMS, TBD (owner to set) % of cases in the pilot area have steps and a business rule, any case runs by id in pre-prod and shows up in its history, and Compliance can export evidence per Jira ticket.

What We're NOT Building

- Loyalty and `rewards-qa` suites - the loyalty runner rejects trigger-service's marker expression and its deployed image is unknown; `rewards-qa` cannot be launched through trigger-service. Later release.

- Manual and exploratory test cases, test plans, release management - not needed to prove the hypothesis.
- Defect tracking - Jira stays the system of record.
- Requirements management - business rules are free text per case in v1.
- SOC 2 control mapping (ENG-GOV-001) - owner deferred it.
- Dropping the old Tcms* tables - separate, later, after a snapshot.
- Changes to existing tables (TestRun, TestCaseResult, TriggerRunHistory, ...) - the app DB user's ALTER rights are unknown.

Success Metrics

Metric	Target	How Measured
Rewards ids imported	100% of ids the scanner finds; every id in the stored collect-only artifact is either found or listed as unresolved	Importer report
Cases with steps and business rule in pilot area	TBD - owner to set % and pick the area	Case list filter <code>has_steps AND has_rule</code>
Run by id	1 demo case runs alone in pre-prod; 10 random unambiguous ids each run exactly their own tests	Run results per <code>test_case_id</code>

Compliance export	One Jira key -> CSV of linked cases with latest result per environment in under 1 minute	Demo
QA adoption	TBD - needs QA headcount; target "every QA engineer created or edited a case within 30 days of launch"	TmCaseVersion.edited_by

Open Questions

- ☐ Pilot area and % target for steps and business rule (hypothesis blank). Suggested pilot: the internal login area.
 - ☐ Is "20 days" 20 working days? This plan assumes working days.
 - ☐ Intern's skill level in Python, SQL and FastAPI is unknown. The plan assumes junior, with an AI pair doing most of the typing.
 - ☐ Which host, port and DB user the deployed pre-prod service uses, and whether that user can `CREATE TABLE`. Sources disagree on what port 3306 is (pre-prod / cloud / production). The DB is `trigger_service` in code, "trigger-service" in the brief.
 - ☐ How trigger-service's own image is built and deployed (Argo / ArgoCD / `infra-helm`). The Helm templates could not be read.
 - ☐ How AI agents authenticate to the JSON API. Default in this PRD: a JSON export behind normal login, bearer token later.
 - ☐ Who fixes ambiguous ids in the test source (QA owners, not the intern).
 - ☐ Who gets the editor and admin roles at launch.
 - ☐ Approval to delete `feat/tcms-m0-m1-audit-pack` on origin (shared repo action).
 - ☐ Do old `Tcms*` tables exist in pre-prod today? Verify read-only before any drop.
-

Users & Context

Primary Users

- **QA engineers** (write and run tests), **developers** (review cases), **QA lead / manager** (track coverage and execution). Whole engineering team can view.
- **Current behavior**: cases in spreadsheets or memory; ids only in test code; runs started by free-text filter.
- **Trigger**: finishing or changing a test; reviewing a ticket; planning a run; being asked for evidence.
- **Success state**: every automated test has one record with steps, the business rule and Jira links, and its run history.

Job to Be Done When I add or change a test, I want it recorded once under a unique id with its steps and the business rule it checks, so I can find it, run it by id, and show auditors the evidence.

Non-Users Compliance reviewers are read-only viewers. No external or customer users.

Solution Detail

Core Capabilities (MoSCoW)

Priority	Capability	Rationale
Must	Unique-id import from rewards test source plus hygiene report	Everything keys on the id
Must	Versioned case records (steps, business rule, expected result, status)	The documentation problem

Must	Jira link and filter by Jira key	"Filtered by Jira ticket" is the vision
Must	Run by id; per-case execution history	Core of the success metric
Must	Roles: viewer (read), editor, admin	Compliance is read-only
Must	Read-only JSON export for AI agents	Owner confirmed
Should	Tamper-evident audit log, audit pack per Jira key	SOC 2 value; stretch if days remain
Could	Manual cases, sign-off flow	Deferred
Won't	Loyalty / rewards-qa, test plans, defect tracking	See above

MVP Scope

Rewards suite only: scanner CLI -> import -> case list and detail with edit -> Jira links -> execution history -> Compliance CSV -> run by id -> JSON export, behind Google login with viewer / editor / admin.

User Flow

QA opens `/tcms`, filters by Jira key, opens a case, reads or edits steps and the business rule, clicks "Run this case", picks an environment, and sees the run in history. Compliance opens a Jira key and downloads the CSV.

Technical Approach

Feasibility: HIGH for the repository, UI, API and reports (existing patterns). MEDIUM for exact run-by-id (touches a second repo and an

Argo image rebuild).

Architecture Notes (file references are from exploration of this repo and platform_svc_test-suite; re-verify a line before relying on it)

- **Stack:** Python 3.12, FastAPI, raw SQL on MySQL via `app/services/db.py` (`db.query`, `db.execute`), Jinja2 server-rendered pages, uv, pytest. Follow the layout `app/routes/*.py`, `app/services/*_store.py`, `app/templates/*.html`.
- **New tables use the prefix `Tm`** (proposed; confirm). Old `Tcms`* tables are left alone. Columns:
 - `TmCase`: `id`, `case_id` (VARCHAR(100) UNIQUE), `platform`, `title`, `area`, `test_type` (`api|grpc|ui`), `source_path`, `source_symbol`, `is_skipped`, `id_status` (`unique|shared|ambiguous`), `substring_unsafe`, `steps`, `business_rule`, `expected_result`, `status` (`draft|active|deprecated`), `version`, `audit` columns.
 - `TmCaseVersion`: `append-only` snapshot per edit; UNIQUE (`case_id`, `version`).
 - `TmCaseJira`: `case_id`, `jira_key`, `link_kind`, `linked_by`, `linked_at`, `removed_at`; UNIQUE (`case_id`, `jira_key`).
 - `TmRoleAssignment`: `email`, `role` (`editor|admin`).
- **Execution history is a read-only join, no new table:**
`TestCaseResult.test_case_id -> TestRun / TriggerRunHistory (job_name, environment, started_at, triggered_by, image_digest) and RunJiraTickets. Status words are Passed | Failed | Error | XFailed | Skipped` (`app/test_outcome.py`).
- **Migrations:** homegrown list `_MIGRATIONS` in `app/services/schema.py` plus `sql/migrations/NNN_name.sql`. `tests/test_migrations.py` enforces contiguous numbering, SQL-file parity and no orphans. New migrations start at 008 with names containing `tm_` so they differ from the old 008..026 `tcms_*` rows that may exist in pre-prod `SchemaMigrations`. **CREATE TABLE only. No ALTER.**
- **Roles:** today any signed-in `vananam.com` user can do anything. TCMS write routes must check the role. Mirror the existing

CANCEL_ADMINS pattern for the first admins (TCMS_ADMINS setting). Any new Settings field must satisfy tests/test_config_reaches_the_pod.py (declare it in a k8s/*.yaml ConfigMap or Secret, or add it to the allowlist with a reason).

- **Importer reads the test source statically (ast). It must never import or run the test suite.** A bare pytest or --collect-only against tests/test/ connects to the reporting DB on port 3306 through conf/test.py (verify-test-pr skill). The scanner is a CLI (scripts/tcms_scan.py --repo <path> --out cases.json) that writes a JSON file; POST /api/tcms/import (admin) loads it. The service never needs access to the test repo.
- **Id resolution rules the scanner must mirror** (platform_svc_test-suite/test_suite/src/reporting_db.py:368-382): the test_case_id pytest param first, else the regex (TC_|GRPC_) \S+ inside the description param. The helper with_test_case_ids(ids, rows) (test_suite/src/utilities/test_case_ids.py) must be handled. Ids the scanner cannot resolve are reported, never guessed.
- **Id status:** unique = one test; shared = same id on several parametrized rows of one function; ambiguous = same id in more than one function or file (run-by-id refused); substring_unsafe = id is a prefix of another id (matters only for the -k fallback).
- **Run by id, service side** (app/models.py TriggerRequest, app/routes/trigger.py:138, app/services/k8s_job.py:61-99, app/services/run_launch.py): add case_ids (max 20, each matching ^[A-Za-z0-9][A-Za-z0-9_.-]{2,99}\$, each present in TmCase, none ambiguous, platform rewards only). test_type comes from the case. Set a new env var TEST_CASE_IDS. **Fallback until the runner change ships:** for exactly one id that is unique and not substring_unsafe, set FILTER to the id. Otherwise refuse with a clear message. The runner's entrypoint.sh passes FILTER through eval, so validate ids server-side.
- **Run by id, runner side (separate repo, separate PR, mentor + AI):** platform_svc_test-suite gets a pytest_collection_modifyitems hook that keeps only items whose extract_test_case_id(item) is in

TEST_CASE_IDS, plus a unit test, and the runner image is rebuilt by the **Argo Workflow** (do not add a GitHub Actions build).
Environment lock: one run per environment per cluster, so a busy environment returns 409 and the UI must say so.

- **UI rules** (tests/test_template_helpers.py, tests/test_favicon.py): every template extends base.html, does not redefine esc or formatDate, and compiles; keep the inline favicon. Form posts return 500 without python-multipart (learned on fix/tcms-forms-and-ci): add it and refresh uv.lock.
- **Jira**: v1 validates the key format `^[A-Z][A-Z0-9]+\d+$` only. No Jira API call, so Jira being down cannot break the TCMS.

Technical Risks

Risk	Likelihood	Mitigation
Intern runs the suite or pytest locally and hits the DB on 3306	M	Local MySQL on 3307 and the local kit only; startup refuses TRIGGER_DEV_AUTH=true with REPORTING_PORT=3306; scanner is static; written rule in Guardrails
Ambiguous or missing ids make run-by-id select the wrong tests	H	Exact-match hook; refuse ambiguous ids; hygiene CSV for QA owners to fix
Runner change needs a second repo and an Argo rebuild	M	Separate phase owned by mentor + AI; fallback keeps v1 usable
Scanner misses dynamically built ids	M	Reconcile against the stored collect-only artifact; unresolved ids listed
App DB user lacks CREATE in pre-prod	L	Verify before rollout; migrations are CREATE-only

Old Tcms* tables and SchemaMigrations rows confuse the new migrations	M	New prefix and tm_ migration names; drop old tables later, after a snapshot
Any signed-in user can edit	H	Role check on every write route; tests prove viewer gets 403
20 days is tight for a junior	M	Phases are small and ordered by value; stretch items last; AI pair writes tests first
Direct TestCaseResult writes by the runner keep test_case_id nullable	M	Report "no id" results separately; never fail the page

Implementation Phases

#	Phase	Description	Status	Parallel	Depends	PRP Plan
0	Cleanup and safety net	Archive and remove old TCMS code; rotate leaked DB password	pending	-	-	-
1	Foundation	Local env, guard, Tm tables, store module	pending	with 3, 8	0	-
2	Case API and roles	CRUD, versions, roles, import endpoint	pending	with 3	1	-

3	Static scanner	scripts/ tcms_scan.py and hygiene report	pending	with 1, 2, 8	0	-
4	Case UI	List, detail, edit pages	pending	with 8	2, 3	-
5	Jira linking and filter	Link cases to Jira keys; filter by key	pending	with 6, 8	4	-
6	Execution history and Compliance CSV	Read-only joins, per-case and per-ticket views	pending	with 5, 8	4	-
7	Run by id (service side)	case_ids, TEST_CASE_IDS, fallback, "Run this case"	pending	with 8	4	-
8	Runner exact-id hook	Test-suite repo hook, Argo image rebuild	pending	with 1-7	0	-
9	JSON export for AI agents	Read-only export of cases, steps, rules, Jira links	pending	with 7	2, 3	-
10	Hardening, rollout, handover	Pre-prod rollout with approval, runbook, QA guide	pending	-	4, 5, 6, 7, 9	-
11	Stretch: audit log and audit pack	Hash-chained events, per-Jira audit pack	pending	-	10	-

Phase Details

Phase 0: Cleanup and safety net (mentor + AI, days 0-1; not the

intern)

- **Goal:** A clean repo and no leaked credential.
- **Scope:** (1) Create a git bundle of `chore/tcms-alignment`, `feat/tcms-m0-m1-audit-pack` and `fix/tcms-forms-and-ci`, plus a tarball of the untracked `docs/tcms-agent-guide.md`, `Claude outputs/`, `local-plans/` and `~/vananam/_tcms_bridge/`, stored outside the repo; verify with `git bundle verify`. (2) Delete the three local branches. Delete `origin/feat/tcms-m0-m1-audit-pack` only after explicit owner approval. (3) Remove the untracked files and `_tcms_bridge`. (4) Rotate the DB password hard-coded in `_tcms_bridge/bridge.py` and move it to the vault. (5) Leave `feat/run-result-summary` (3 unpushed commits), `master` and every `Tcms*` DB table untouched. (6) Write the old-table cleanup list into the runbook for later.
- **Success signal:** `git branch -a` shows no `tcms` branch; `git status` is clean apart from intended work; `uv run pytest -q` passes offline; the bundle verifies; the old password no longer works.

Phase 1: Foundation (days 1-3)

- **Goal:** A local app with the new schema.
- **Scope:** `.env` pointing at local MySQL (3307) with `TRIGGER_DEV_AUTH=true`; startup guard that refuses dev-auth against port 3306 (with a test); migrations 008. . creating `TmCase`, `TmCaseVersion`, `TmCaseJira`, `TmRoleAssignment` in both `_MIGRATIONS` and `sql/migrations/`; `app/services/tcms_store.py` with unit tests on the `fake_db` double. Tests first.
- **Success signal:** `uv run pytest -q` passes including `test_migrations.py`; the app starts locally, and the four tables exist in the local DB only.

Phase 2: Case API and roles (days 3-5)

- **Goal:** A safe write path.
- **Scope:** `app/routes/tcms.py` with `GET/PATCH /api/tcms/cases`, `GET /api/tcms/cases/{case_id}`, `GET .../`

versions, POST /api/tcms/import (admin); every edit writes a TmCaseVersion; role checks (viewer / editor / admin) with TCMS_ADMINS bootstrap; register the router in app/main.py.

- **Success signal:** tests show viewer gets 403 on every write, editor can edit, admin can import, and an edit creates version N+1.

Phase 3: Static scanner (days 3-7)

- **Goal:** Turn the rewards test source into case records without running anything.
- **Scope:** scripts/tcms_scan.py using ast; output contract below; hygiene report (duplicates, ambiguous, substring_unsafe, missing_id, unresolved) as JSON and CSV; reconcile against the stored collect-only artifact.
- **Output contract:**

```
{"source": {"repo", "commit", "scanned_at"}, "cases": [{"case_id", "platform": "rewards", "test_type", "area", "s {...}].
```
- **Success signal:** on the current rewards suite the scan finds every id in the collect-only artifact or lists it as unresolved; the scan never imports test modules or opens a DB connection (test asserts this).

Phase 4: Case UI (days 6-10)

- **Goal:** QA can read and edit cases.
- **Scope:** /tcms list with filters (text, area, status, Jira key); /tcms/cases/{case_id} detail with steps, business rule, expected result, version history; edit form for editors; viewers see no edit controls; add python-multipart; nav link in base.html.
- **Success signal:** template tests pass; a manual walkthrough as viewer and as editor; a screenshot per page attached to the PR.

Phase 5: Jira linking and filter (days 9-11)

- **Goal:** "Filtered by Jira ticket".
- **Scope:** link and unlink (soft delete via removed_at), key format validation, filter by key, bulk link from a pasted list of ids.
- **Success signal:** tests for validation, idempotent re-link, and filter results.

Phase 6: Execution history and Compliance CSV (days 11-14)

- **Goal:** Evidence per case and per ticket.
- **Scope:** GET `/api/tcms/cases/{case_id}/executions`; GET `/api/tcms/jira/{jira_key}/report` and `.csv` (linked cases, latest result per environment, run, date, who triggered, image digest); case page shows the history; results with no `test_case_id` are reported separately.
- **Success signal:** tests against seeded `TestCaseResult` rows; the CSV for one real Jira key opens and matches the UI.

Phase 7: Run by id, service side (days 13-17)

- **Goal:** Run one case in isolation.
- **Scope:** as in Technical Approach: `TriggerRequest.case_ids`, validation, `TEST_CASE_IDS` env var, single-id `FILTER` fallback, refusal messages, "Run this case" button, 409 handling.
- **Success signal:** unit tests for every refusal path; a local run through the local kit executes exactly one case; then one pre-prod run, after approval.

Phase 8: Runner exact-id hook (mentor + AI, days 4-12, separate repo)

- **Goal:** Exact selection by id.
- **Scope:** hook in `platform_svc_test-suite`, unit test (hermetic), PR reviewed against that repo's `CLAUDE.md`; Argo rebuild of `service/test-runner:latest`; confirm the new pod `imageID` digest.
- **Success signal:** an id with a prefix collision runs only its own tests; a missing id runs nothing and fails loudly.

Phase 9: JSON export for AI agents (days 16-17)

- **Goal:** Business rules readable by AI.
- **Scope:** GET `/api/tcms/export.json` (cases, steps, business rule, expected result, Jira links, id status) behind the normal login; a documented script to dump it to a file for local agents. Bearer token is out of scope until the open question is answered.
- **Success signal:** the file validates against a documented schema and includes every active case.

Phase 10: Hardening, rollout, handover (days 18-20)

- **Goal:** Live in pre-prod and usable by QA.
- **Scope:** owner approval; verify DB grants; deploy through the normal Helm / Argo path (mentor); smoke checklist; runbook; one-page "how to add a test case" guide; import the real cases; hygiene CSV handed to QA owners.
- **Success signal:** QA lead completes the demo script: filter by Jira key, open a case, run it by id, export the CSV.

Phase 11: Stretch - audit log and audit pack

- **Goal:** Tamper-evident history.
- **Scope:** `TmAuditEvent` with a hash chain and a verify endpoint; per-Jira-key audit pack.
- **Success signal:** tampering with one row fails the chain verification test.

Parallelism Notes

The intern works in numeric order. Phases 1, 3 and 8 are independent and can run at once if the mentor or AI take 3 and 8. After Phase 4, phases 5, 6 and 7 are independent of each other. Phase 8 must land before the fallback is removed, but Phase 7 ships without it.

Guardrails for the intern and the AI pair

1. **Local only.** Use the local MySQL on 3307 and `~/vananam/local-dev/trs-api`. Never point anything at port 3306. Never run `main.py` or bare `pytest` against `tests/test/`, and never use `--seeder true`. Pre-prod only through the deployed service, after owner approval.
2. **One phase, one PR.** Tests first, then code. `uv run pytest -q` must pass offline before every push.
3. **Match the surrounding code:** store modules, routers and templates as listed above. No new dependency without a reason

- in the PR (`python-multipart` is the known exception).
4. **Migrations are CREATE-only**, in `_MIGRATIONS` and `sql/migrations/` together, numbered contiguously.
 5. **Do not edit old tables or the runner's writes** to `TestRun / TestCaseResult`.
 6. **No AI co-author or session trailers** in commits or PR descriptions. Origin's `ci/pytest-and-commit-hygiene` fails PRs that contain them.
 7. **Ask before shared actions**: deleting the origin branch, any pre-prod change, any cluster command.
-

Decisions Log

Decision	Choice	Alternatives	Rationale
Build or adopt	Build in trigger-service	Kiwi TCMS (free, self-host), Qase (free for 3 users), TestRail (\$37-\$74/user/month), Zephyr / Xray (per Jira seat)	Owner decision; run-by-id and execution evidence already live in trigger-service; cost-effective
Old build	Archive then delete code; leave DB tables	Keep and extend	Owner: start from scratch
Reuse	Ideas only: versioned cases, hash-chained audit, id-hygiene lessons	Import the 3,565 seeded cases	Seeded rows came from run history, 617 had the wrong platform, 46 had steps
Table names	Prefix <code>Tm</code>	Reuse <code>Tcms</code> *	Old tables remain in the DB

Execution data	Read-only join on existing tables	New execution table	No ALTER rights known; no duplication
Import method	Static ast scan to JSON, then upload	pytest --collect-only	Collect-only connects to the reporting DB
Id policy	Keep existing ids; exact-match selection; refuse ambiguous	Renumber everything	Existing results and history key on these ids
Suites in v1	Rewards only	Loyalty, rewards-qa	Runner problems and no launch path
Jira	Key format check only	Live Jira validation	Avoids a Jira outage breaking the TCMS
Environment	Local only until approval	Dev against pre-prod	Owner decision and safety
Roles	viewer / editor / admin	Single level (today's behaviour)	Compliance is read-only
AI access	JSON export behind login	Bearer token now	Token auth is an open question

Research Summary

Market Context Kiwi TCMS is free, GPLv2, self-hosted, with a pytest reporter and Jira sync. TestLink has requirement linkage. Qase has a free tier for 3 users. TestRail is \$37 (Professional) or \$74 (Enterprise)

per user per month. Zephyr and Xray live inside Jira and are priced by Jira seats. Every tool links automated tests to cases through an id annotation and lets teams run selected cases by those ids. For SOC 2 and ISO 27001, auditors want immutable, dated evidence across the observation period (6-12 months for SOC 2 Type II) and a requirement -> test -> result trail. Sources: keploy.io "Best Open Source Test Management Tools in 2026"; getautonoma.com "Test Case Management Tools: Buyer-Neutral 2026 Comparison"; qase.io "Requirements Traceability in Regulated Industries"; testrail.com "Pytest Test Management with TestRail Integration"; konfirmity.com "SOC 2 Evidence Requirements (2026)".

Technical Context

- trigger-service: Python 3.12 / FastAPI / raw-SQL MySQL / Jinja2, one replica, Google OIDC, about 550 offline tests; no CI on `master`; image build and deploy path unknown from the repo.
- The runner writes `TestRun` and `TestCaseResult` straight to MySQL (`reporting_db.py`); `test_case_id` is `VARCHAR(100)`, nullable.
- Selection today: `-m<type>` plus `-k "<filter>"` (substring over node ids). One active run per environment per cluster (K8s Lease).
- Old TCMS: 14 `Tcms*` tables and migrations 008-026 exist on branches only; live DB state unverified. `_tcms_bridge/bridge.py` hard-codes a DB credential.
- Not verified: contents of the Helm `deployment.yaml` and `_helpers.tpl`; DB grants; which environment port 3306 is.

Generated: 2026-10-05 Status: DRAFT - needs validation